Projet: Linux Administration and Docker Orchestration

Objective: To master Linux administration and Docker orchestration

Technical Challenge: Bash system management automation, CPU scheduling simulation using SRTF and Round Robin, and Docker deployment of a MySQL and Web multi-service stack.

Prepared for: Dr. EL BENANY Mohamed Mahmoud - LIU-2026

Slogan: Move from theory to action.

Deliverable	Content
Git repository	Bash, Python, Docker files
PDF report	Explanation, algorithms, steps, results
Video demo	3 minutes: run Bash, Python, Docker

1. Introduction

This project transforms operating system concepts into practical activities. It combines administration automation, CPU scheduling analysis, and service deployment using containers.

The Bash part automates system management tasks. The CPU part simulates how an operating system chooses which process receives the processor. The Docker part deploys a web application connected to a MySQL database.

Project objectives

The main objectives are: understand Linux commands, automate administration tasks, compare SRTF and Round Robin scheduling, deploy services with Docker Compose, and organize the work in a clean Git repository.

Tools used

Tool	Role
Bash	Automate Linux system management
Python	Simulate CPU scheduling algorithms
Docker Compose	Deploy MySQL and Web services
Git	Version control and collaboration

2. Bash: System Management Automation

Bash is a command language used in Linux. Instead of writing commands one by one, we create a script that executes them automatically. In this project, the script collects system information and saves it in a report file.

Tasks automated

The script creates a reports directory, collects hostname, current user, uptime, operating system information, CPU usage, memory usage, disk usage, top processes, and network information.

Main Bash script

```
#!/bin/bash
REPORT_DIR="reports"
REPORT_FILE="$REPORT_DIR/system_report_$(date +%Y%m%d_%H%M%S).txt"
mkdir -p "$REPORT_DIR"
{
  echo "SYSTEM MANAGEMENT REPORT"
  echo "Date: $(date)"
  echo "Hostname: $(hostname)"
  echo "Current user: $(whoami)"
  echo "Uptime: $(uptime -p 2>/dev/null || uptime)"
  free -h
  df -h
  ps aux --sort=-%cpu | head -10
} > "$REPORT_FILE"
echo "Report generated successfully: $REPORT_FILE
```

How to run it

```
cd bash
chmod +x system_management.sh
./system_management.sh
```

3. Bash Script Explanation

The command mkdir -p creates the report folder only if it does not already exist. The variable REPORT_FILE contains the output file name with the current date and time. This avoids replacing old reports.

The block between braces { ... } groups all outputs and redirects them into one file using >. Commands such as free -h, df -h, ps aux, top, ip addr and cat /etc/os-release are used to describe the current state of the system.

Expected result

After execution, the user obtains a text file in the reports folder. This file can be included as evidence in the Git repository or shown during the video demo.

Why this is useful

A system administrator often needs to monitor resources and collect diagnostic information. Automation saves time, reduces errors, and makes the task repeatable.

4. CPU Scheduling: Concept

CPU scheduling is the mechanism used by the operating system to choose which process runs on the processor. Each process has an arrival time and a burst time. Arrival time means when the process becomes ready. Burst time means how long it needs the CPU.

Metrics

Metric	Meaning
Completion time	Time when the process finishes
Turnaround time	Completion time - Arrival time
Waiting time	Turnaround time - Burst time
Average waiting time	Mean waiting time for all processes

Dataset used in the simulation

Process	Arrival	Burst
P1	0	5
P2	1	3
P3	2	8
P4	3	6

5. SRTF Algorithm

SRTF means Shortest Remaining Time First. It is a preemptive scheduling algorithm. At each unit of time, the CPU selects the available process with the smallest remaining time. If a new process arrives with a shorter remaining time, the current process can be interrupted.

Python idea

The program keeps a remaining time for each process. At every time unit, it filters available processes and selects the one with the minimum remaining time.

```
available = [p for p in processes if p['arrival'] <= time and remaining[p['pid']] > 0]
current = min(available, key=lambda p: remaining[p['pid']])
remaining[current['pid']] -= 1
time += 1
```

Advantages and limits

SRTF gives good results for short processes because they finish quickly. However, long processes may wait too much if many short processes arrive.

6. Round Robin Algorithm

Round Robin gives each process the CPU for a fixed time called quantum. If the process is not finished after this time, it returns to the ready queue. This method is fair because every process gets a turn.

Python idea

```
ready = deque()
current = ready.popleft()
run_time = min(quantum, remaining[pid])
time += run_time
remaining[pid] -= run_time
if remaining[pid] > 0:
    ready.append(current)
```

Quantum

The quantum strongly affects the result. A very small quantum creates many context switches. A very large quantum makes Round Robin behave almost like FCFS.

Advantages and limits

Round Robin is simple and fair. It is suitable for interactive systems, but average waiting time can be higher than SRTF.

7. SRTF vs Round Robin Comparison

Criterion	SRTF	Round Robin
Decision rule	Smallest remaining time	Fixed quantum and queue
Preemption	Yes	Yes
Fairness	Medium	High
Good for	Short processes	Interactive systems
Risk	Starvation of long jobs	Many context switches

What to show in the demo

Run round_robin.py and srtf.py, show the Gantt chart, show waiting time and turnaround time, then explain which algorithm is more suitable depending on the goal: performance for short jobs or fairness between processes.

8. Docker Multi-Service Stack

Docker Compose is used to deploy two services: a MySQL database and a Python Flask web application. The web service connects to the database using environment variables. This demonstrates orchestration of multiple services.

docker-compose.yml

```
services:
  db:
    image: mysql:8.0
    environment:
      MYSQL_ROOT_PASSWORD: root_password_fst
      MYSQL_DATABASE: liu_db
    ports:
      - "3306:3306"
  web:
    build: ./web
    depends_on:
      - db
    ports:
      - "5000:5000
```

Run commands

```
cd docker
docker compose up -d --build
docker compose ps
# Browser: http://localhost:5000
docker compose down
```

9. Organization, Video Demo and Conclusion

Suggested group organization

Student	Task
Student 1	Bash automation and report screenshots
Student 2	Python CPU scheduling: SRTF and Round Robin
Student 3	Docker Compose stack and Git repository organization

3-minute video plan

Minute 1: explain objective and run Bash script. Minute 2: run Python simulations and compare results. Minute 3: run Docker Compose and open the web application in the browser.

Conclusion

This project connects theory with practice. Bash demonstrates automation, Python demonstrates CPU scheduling behavior, and Docker demonstrates deployment of real services. The final result is a clean, reproducible and demonstrable system administration project.

Repository structure

```
project/
  bash/system_management.sh
  python/round_robin.py
  python/srtf.py
  docker/docker-compose.yml
  docker/web/Dockerfile
  docs/LIU2026_Report.pdf
  README.md
```